KERN-LITE

Complete Student Project Handbook

Phase 0 setup + seven-day implementation | Teams of 2-3

Configure the STM32 project, establish a tested build baseline, then implement firmware, protocol, sensor processing, circular storage, a Python ground station, analytics, recovery, and validation.

Every phase ends with evidence: host tests, Python tests, hardware demonstrations, fault injection, or documented artifacts.

Phase 0 and every daily gate are mandatory. Do not continue until the current gate passes.

PROJECT OVERVIEW

KERN-LITE begins with a mandatory Phase 0 setup gate and continues through seven consecutive implementation phases. Each phase produces a working, testable increment that becomes the foundation for the next phase.

CANONICAL SPECIFICATION This handbook is the single source of truth for the project. Follow its folder names, APIs, protocol layout, constants, and daily gates.

Learning outcomes

- Configure a reproducible STM32CubeIDE, CMSIS-RTOS2, HAL, and FatFs project baseline.
- Design a byte-oriented binary protocol with CRC-32, framing, decoding, and resynchronization.
- Integrate UART communication with STM32, FreeRTOS tasks, interrupts, and synchronization.
- Build a sensor pipeline with filtering, thresholds, hysteresis, validation, and fault reporting.
- Implement persistent circular file storage with metadata, replay, wraparound, and crash recovery.
- Develop a Python ground station for commands, telemetry, integrity checks, sessions, and analytics.
- Validate the system through unit tests, live integration tests, fault injection, and exports.

How to use this handbook

- Complete Phase 0 before beginning protocol or application implementation.
- Begin each day by assigning the listed tasks to the appropriate team members.
- Treat every task identifier as a deliverable and commit working increments frequently.
- Run the required tests during implementation, not only at the end of the day.
- Prepare the evidence listed in each End-of-Day Review.
- Do not begin the next phase until the blocker gate passes.

BLOCKER RULE A failed gate blocks all downstream work. Fix the root cause before continuing.

Project roadmap

Phase	Focus	Primary outcome
0	Project setup	CubeIDE, CMSIS-RTOS2, peripherals, scaffold, scheduler smoke test
1	Protocol foundations	CRC-32, frame codec, cross-language vectors
2	UART round-trip	Real board communication and command/status exchange
3	Sensors and DSP	10 Hz SensorRecord pipeline and live telemetry
4	Circular storage	Multi-file ring, replay, metadata, and recovery
5	FSM and commands	Full command cycle and RTOS orchestration
6	GS analytics	Charts, statistics, timeline, alerts, quality, exports
7	Validation and demo	Fault injection, final artifacts, and full demonstration

Team roles

Role	Lead area	Primary responsibility	Two-person team
Member A	Firmware lead	C/C++, STM32, FreeRTOS	Person 1
Member B	Ground station lead	Python, serial link, models, UI/analytics	Person 2
Member C	Testing and integration	Cross-cutting tests, hardware	Split between A and B

Role	Lead area	Primary responsibility	Two-person team
		integration, evidence	

Canonical repository structure

```
kern-lite/
|-- kern-lite.ioc
|-- Core/                  # CubeMX-generated code
|-- Drivers/               # generated; commit
|-- Middlewares/           # generated; commit
|-- firmware/
|   |-- hal/
|   |-- sensors/
|   |-- dsp/
|   |-- protocol/
|   |-- storage/
|   |-- recorder/
|   `-- system/
|-- groundstation/
|-- tests/
|   |-- host/              # firmware logic tests; run on PC
|   `-- gs/                # pytest tests
|-- docs/
|   `-- README.md
`-- .gitignore
```

Handbook navigation

Phase 0 | Day 1 | Day 2 | Day 3 | Day 4 | Day 5 | Day 6 | Day 7

PHASE 0

PROJECT SETUP AND SMOKE TEST

PHASE OBJECTIVE Create a reproducible CubeIDE project that builds cleanly, runs the scheduler, drives the heartbeat LED, prints over UART2, and contains the complete source scaffold required by Days 1-7.

IMPORTANT CORRECTIONS Select CMSIS-RTOS2 (CMSIS_V2). When CubeIDE asks whether to initialize peripherals with their default modes, choose Yes; then explicitly verify and override every pin according to the fixed pin table below.

0.1 Exit criteria

- [ ] The project builds with zero errors and zero warnings.
- [ ] The FreeRTOS scheduler starts on the NUCLEO-L476RG.
- [ ] LED1_BLUE on PC9 toggles once per second from the System task.
- [ ] UART2 prints KERN-LITE ALIVE once per second during the Phase 0 smoke test.
- [ ] The watchdog is refreshed by the System task and resets the board if task execution stalls.
- [ ] Every required source file and test folder exists in the canonical repository layout.
- [ ] Both team members can clone, build, flash, and observe the same smoke-test behavior.

0.2 Prerequisites

- STM32CubeIDE installed using the course-approved version.
- NUCLEO-L476RG connected through the ST-Link USB connector.
- A serial terminal capable of 115200 baud, 8 data bits, no parity, 1 stop bit.
- A FAT32-formatted microSD card for later phases; the card is optional for the Phase 0 smoke test.
- A shared Git repository with push access for every team member.

0.3 Fixed board pin map

Pin	Signal	Mode	Notes
PA2	USART2_TX	AF7 / USART	ST-Link virtual COM TX
PA3	USART2_RX	AF7 / USART	ST-Link virtual COM RX
PA5	SPI1_SCK	AF5 / SPI	SD clock; override the board-default LED assignment
PA6	SPI1_MISO	AF5 / SPI	SD MISO
PA7	SPI1_MOSI	AF5 / SPI	SD MOSI
PB6	SD_CS	GPIO output	Initial level HIGH
PB5	DHT_DATA	GPIO open-drain	Initial HIGH; external pull-up required
PA0	ADC1_IN5	Analog	Potentiometer
PA1	ADC1_IN6	Analog	Photodiode
PA4	ADC1_IN9	Analog	LM35 temperature
PB4	TIM3_CH1	AF2 / timer	Buzzer PWM
PC9	LED1_BLUE	GPIO output	Heartbeat LED
PB8	LED2_RED	GPIO output	Fault LED
PC7	RGB_R	GPIO output	RGB red
PC6	RGB_G	GPIO output	RGB green
PC8	RGB_B	GPIO output	RGB blue
PA10	SW1	GPIO input	Pull-up
PB3	SW2	GPIO EXTI3	Pull-up, falling edge, IRQ priority 5
PC13	B1	GPIO input	Nucleo onboard button

0.4 Create the CubeIDE project

Step	Action	Required setting
1	File > New > STM32 Project	Select the NUCLEO-L476RG board.
2	Project name	Use kern-lite and keep the project inside the team repository.
3	Targeted language	Select C++. Generated HAL files may remain C; application files under firmware/ use C++.
4	Project type	Select STM32Cube.
5	Initialize default peripherals	Choose Yes. Afterwards verify every assignment and replace conflicting defaults, especially PA5.
6	Open the .ioc	Keep kern-lite.ioc under version control.

0.5 Clock configuration - 80 MHz

Item	Setting
RCC	HSE = Bypass Clock Source; LSE disabled unless the board design requires it.
PLL source	HSE, 8 MHz from the ST-Link MCO.
PLL	PLLM=1, PLLN=20, PLLR=2 -> 80 MHz SYSCLK.
Bus clocks	HCLK, APB1, and APB2 configured from the 80 MHz system clock.
HAL timebase	TIM1. SysTick remains available to FreeRTOS/CMSIS-RTOS2.
Verification	No red clock-tree errors; SystemCoreClock resolves to 80,000,000 Hz.

0.6 Peripheral configuration

USART2

Parameter	Setting
Mode	Asynchronous
Pins	PA2 TX, PA3 RX
Format	115200, 8 data bits, no parity, 1 stop bit, no flow control
Interrupt	Disabled in Phase 0; enabled for interrupt-driven RX in Day 2

SPI1 and SD chip select

Parameter	Setting
Mode	Full-duplex master; software-controlled NSS
Pins	PA5 SCK, PA6 MISO, PA7 MOSI
SPI mode	8-bit, CPOL low, CPHA first edge
Initialization speed	Prescaler /256 at 80 MHz -> 312.5 kHz, below the SD-card initialization limit
Run speed	The disk driver may switch to /4 only after the card completes initialization
SD_CS	PB6 output, initial HIGH

ADC1

Parameter	Setting
Inputs	IN5/PA0, IN6/PA1, IN9/PA4, single-ended
Resolution	12-bit, right aligned
Conversion mode	Single conversion; scan and continuous modes disabled
Sampling time	47.5 cycles for each selected channel
Clock	Synchronous clock divided by 4
Implementation note	hal::adc::read() selects and converts one channel at a time in Day 3

Timers and watchdog

Peripheral	Setting
TIM3 CH1	PWM on PB4; prescaler 79, initial period 999
TIM6	Free-running 1 us counter; prescaler 79, ARR 65535; used by the DHT11 driver
IWDG	Prescaler /256, reload 499 -> approximately 4 s with a 32 kHz LSI

GPIO and interrupt details

- All LEDs and RGB outputs start LOW.
- DHT_DATA starts HIGH and must not hold the DHT11 bus low during boot.
- SW1 uses a pull-up. SW2 uses a pull-up and falling-edge EXTI3.
- Enable EXTI3_IRQn, not EXTI line[9:5], and set preemption priority 5.
- PC13 uses the Nucleo board circuitry; do not add an internal pull unless required by the board revision.

0.7 FreeRTOS and CMSIS-RTOS2

Parameter	Setting
Interface	CMSIS_V2
Preemption	Enabled
Static allocation	Enabled
Mutexes	Enabled
Software timers	Enabled
Heap scheme	heap_4
Total heap	20,480 bytes as a starting value; verify actual use
Default task	Delete it; the project creates Sensor, Storage, Comms, and System tasks
Useful APIs	Enable vTaskDelay, vTaskDelayUntil, and uxTaskGetStackHighWaterMark
Newlib reentrancy	Disabled unless the toolchain configuration explicitly requires it

API RULE The project is configured through CMSIS-RTOS2. Where this handbook explicitly requires a native FreeRTOS static API such as xTaskCreateStatic or xSemaphoreCreateMutexStatic, that direct API is permitted.

0.8 FatFs

Parameter	Setting
Mode	User-defined disk I/O
Sector size	512 bytes

Parameter	Setting
Long file names	Disabled; project filenames use 8.3 format
Read-only	Disabled
RTC	Disabled for this project
Disk adapter	Integrate the course-provided SPI SD disk I/O adapter; do not use a fixed fake sector count
Phase 0 test	Compilation is mandatory; mounting the card is optional until storage integration

SD DRIVER REQUIREMENTS The adapter must initialize below 400 kHz, issue CMD58 to distinguish SDHC from SDSC, use block addressing correctly, support every sector count passed by FatFs, and obtain capacity from the card CSD rather than a hard-coded placeholder.

0.9 Generate code safely

- Generate code from the .ioc file and build once before adding application files.
- Keep generated main.c and generated HAL sources as C. Do not rename main.c; this avoids duplicate or regenerated entry files.
- Place application C++ in firmware/. Use an extern "C" bridge function named kern_boot().
- In the generated RTOS source file (app_freertos.c or freertos.c), call kern_boot() inside the USER CODE section of MX_FREERTOS_Init().
- In a C source file, declare the bridge as extern void kern_boot(void);. The extern "C" declaration belongs in the C++ implementation.
- Never edit generated code outside USER CODE guards.

```
/* app_freertos.c or freertos.c - inside a USER CODE section */
extern void kern_boot(void);

void MX_FREERTOS_Init(void)
{
    /* USER CODE BEGIN RTOS_THREADS */
    kern_boot();
    /* USER CODE END RTOS_THREADS */
}
```

0.10 Create the canonical source scaffold

```
firmware/
|-- hal/
|   |-- gpio.hpp
|   |-- adc.hpp
|   |-- watchdog.hpp
|   |-- pwm.hpp
|   |-- tone.hpp / tone.cpp
|   `-- exti.hpp / exti.cpp
|-- sensors/
|   |-- lm35.hpp
|   |-- photodiode.hpp
|   |-- potentiometer.hpp
|   |-- dht11.hpp / dht11.cpp
|   |-- radiation_latch.hpp / radiation_latch.cpp
|   `-- buttons.hpp / buttons.cpp
|-- dsp/
|   |-- moving_average.hpp
|   |-- threshold_detector.hpp
|   `-- channel.hpp
|-- protocol/
|   |-- frame.hpp
|   |-- crc32.hpp / crc32.cpp
|   `-- codec.hpp / codec.cpp
|-- storage/
|   |-- sensor_record.hpp
|   `-- circular_log.hpp / circular_log.cpp
|-- recorder/
|   |-- sensor_bus.hpp
|   |-- state_machine.hpp / state_machine.cpp
|   |-- comm_link.hpp / comm_link.cpp
|   `-- command_handler.hpp / command_handler.cpp
`-- system/
    |-- board.hpp
```

```
    |-- config.hpp
    |-- tasks_defs.hpp
    |-- orchestrator.hpp / orchestrator.cpp
    |-- tasks.hpp / tasks.cpp
    `-- init.hpp / init.cpp
```

- Add firmware/ and its subfolders as source locations and include paths in CubeIDE.
- Create groundstation/, tests/host/, tests/gs/, and docs/ at repository root.
- Do not create a separate comms/ folder; communication classes live in firmware/recorder/.
- All stubs must match the final APIs shown below so later phases replace behavior without renaming interfaces.

0.11 Core compile-time contracts

system/board.hpp

```
#pragma once
#include "stm32l4xx_hal.h"
#include <cstdint>

namespace kern::board {
struct Pin { GPIO_TypeDef* port; uint8_t n; };
inline constexpr Pin LED1_BLUE{GPIOC, 9};
inline constexpr Pin LED2_RED {GPIOB, 8};
inline constexpr Pin RGB_R    {GPIOC, 7};
inline constexpr Pin RGB_G    {GPIOC, 6};
inline constexpr Pin RGB_B    {GPIOC, 8};
inline constexpr Pin SD_CS    {GPIOB, 6};
inline constexpr Pin DHT_DATA {GPIOB, 5};
inline constexpr Pin SW1      {GPIOA, 10};
inline constexpr Pin SW2      {GPIOB, 3};
inline constexpr Pin B1       {GPIOC, 13};

enum class AdcChannel : uint32_t {
    Pot = ADC_CHANNEL_5,
    Photodiode = ADC_CHANNEL_6,
    Lm35 = ADC_CHANNEL_9,
};
}
```

system/config.hpp and tasks_defs.hpp

```
// config.hpp
#pragma once
#include <cstdint>
namespace kern::config {
inline constexpr uint32_t kUartBaud = 115200;
inline constexpr uint32_t kSensorPeriodMs = 100;
inline constexpr uint32_t kStoragePeriodMs = 100;
inline constexpr uint32_t kCommsPeriodMs = 10;
inline constexpr uint32_t kSystemPeriodMs = 50;
inline constexpr uint32_t kMetaFlushEveryN = 16;
inline constexpr uint8_t  kLogFileCount = 4;
inline constexpr uint16_t kRecordsPerFile = 256;
inline constexpr uint32_t kEraseMagic = 0xDEADC0DEu;
}

// tasks_defs.hpp
#pragma once
#include "FreeRTOS.h"
#include "task.h"
namespace kern::tasks {
inline constexpr uint32_t kSensorStack = 512;
inline constexpr uint32_t kStorageStack = 768;
inline constexpr uint32_t kCommsStack = 384;
inline constexpr uint32_t kSystemStack = 384;
inline constexpr UBaseType_t kSensorPrio = tskIDLE_PRIORITY + 2;
inline constexpr UBaseType_t kStoragePrio = tskIDLE_PRIORITY + 2;
inline constexpr UBaseType_t kCommsPrio = tskIDLE_PRIORITY + 3;
inline constexpr UBaseType_t kSystemPrio = tskIDLE_PRIORITY + 1;
}
```

hal wrappers

```
// gpio.hpp
#pragma once
#include "../system/board.hpp"
namespace kern::hal::gpio {
inline uint16_t mask(board::Pin p) { return static_cast<uint16_t>(1u << p.n); }
```

```
inline void set(board::Pin p) { HAL_GPIO_WritePin(p.port, mask(p), GPIO_PIN_SET); }
inline void clear(board::Pin p) { HAL_GPIO_WritePin(p.port, mask(p), GPIO_PIN_RESET); }
inline void toggle(board::Pin p) { HAL_GPIO_TogglePin(p.port, mask(p)); }
inline bool read(board::Pin p) { return HAL_GPIO_ReadPin(p.port, mask(p)) == GPIO_PIN_SET; }
}

// adc.hpp - Phase 3 implements channel selection and conversion
#pragma once
#include "../system/board.hpp"
#include <cstdint>
namespace kern::hal::adc {
inline uint16_t read(board::AdcChannel ch) { (void)ch; return 0; }
inline float toVolts(uint16_t raw) { return (static_cast<float>(raw) / 4095.0f) * 3.3f; }
}

// watchdog.hpp
#pragma once
#include "stm32l4xx_hal.h"
namespace kern::hal::watchdog {
inline void kick(IWDG_HandleTypeDef& h) { HAL_IWDG_Refresh(&h); }
}
```

protocol/frame.hpp

```
#pragma once
#include <cstdint>
#include <cstddef>
namespace kern::protocol {
inline constexpr uint8_t kStx = 0xAB;
inline constexpr uint8_t kEtx = 0xCD;
inline constexpr size_t kMaxPayload = 256;
inline constexpr size_t kFrameOverhead = 8;

enum class FrameType : uint8_t {
    CmdStart  = 0x01,
    CmdStop   = 0x02,
    CmdStatus = 0x03,
    CmdReplay = 0x04,
    CmdErase  = 0x06,
    Status    = 0x10,
    Record    = 0x11,
    Ack       = 0x20,
    Nack      = 0x21,
};

enum class NackCode : uint8_t {
    BadCommand = 1,
    InvalidState = 2,
    BadLength = 3,
    BadMagic = 4,
    StorageError = 5,
};

struct Frame {
    FrameType type{};
    uint8_t payload[kMaxPayload]{};
    uint16_t len = 0;
};
}
```

protocol/crc32.* and codec.* stubs

```
// crc32.hpp
#pragma once
#include <cstdint>
#include <cstddef>
namespace kern::protocol {
uint32_t crc32(const uint8_t* data, size_t len);
uint32_t crc32Begin();
uint32_t crc32Update(uint32_t crc, const uint8_t* data, size_t len);
uint32_t crc32Finalize(uint32_t crc);
}

// crc32.cpp - Day 1 replaces these stubs
#include "crc32.hpp"
namespace kern::protocol {
uint32_t crc32(const uint8_t*, size_t) { return 0; }
uint32_t crc32Begin() { return 0xFFFFFFFFu; }
uint32_t crc32Update(uint32_t crc, const uint8_t*, size_t) { return crc; }
uint32_t crc32Finalize(uint32_t crc) { return crc ^ 0xFFFFFFFFu; }
}
```

```
// codec.hpp
#pragma once
#include "frame.hpp"
namespace kern::protocol {
enum class DecodeResult { NeedMore, FrameReady, CrcError, SyncError };
size_t encode(const Frame& f, uint8_t* outBuf, size_t outSize);
class Decoder {
public:
    DecodeResult feed(uint8_t byte);
    const Frame& frame() const { return m_frame; }
    void reset();
private:
    Frame m_frame{};
};
}

// codec.cpp - Day 1 implements the state machine
#include "codec.hpp"
namespace kern::protocol {
size_t encode(const Frame&, uint8_t*, size_t) { return 0; }
DecodeResult Decoder::feed(uint8_t) { return DecodeResult::NeedMore; }
void Decoder::reset() { m_frame = {}; }
}
```

storage/sensor_record.hpp

```
#pragma once
#include <cstdint>
#include <cstddef>
namespace kern::storage {
#pragma pack(push, 1)
struct SensorRecord {
    uint32_t timestamp;
    uint16_t ms;
    uint16_t seq;
    int16_t lm35_c;
    int16_t dht_temp_c;
    uint16_t dht_hum;
    uint16_t light;
    uint16_t pot;
    uint8_t alert_bits;
    uint8_t state;
    uint8_t fault_bits;
    uint8_t reserved[7];
    uint32_t crc32;
};
#pragma pack(pop)
static_assert(sizeof(SensorRecord) == 32);
static_assert(offsetof(SensorRecord, crc32) == 28);

inline constexpr uint8_t kFaultLm35Range = 0x01;
inline constexpr uint8_t kFaultDhtTimeout = 0x02;
inline constexpr uint8_t kFaultDhtBadData = 0x04;
inline constexpr uint8_t kFaultLightStuck = 0x08;
inline constexpr uint8_t kFaultPotStuck = 0x10;
}
```

recorder and storage stubs

```
// state_machine.hpp
#pragma once
#include <cstdint>
namespace kern::recorder {
enum class State : uint8_t { Idle=0, Recording=1, Fault=2 };
enum class Event : uint8_t { ChecksPassed, SdFault, UartStart, UartStop, ShortPress, FaultCleared };
class StateMachine {
public:
    bool process(Event);
    State state() const { return m_state; }
    bool isIdle() const { return m_state == State::Idle; }
    bool isLogging() const { return m_state == State::Recording; }
    bool isFault() const { return m_state == State::Fault; }
private:
    State m_state = State::Idle;
};
}

// state_machine.cpp
#include "state_machine.hpp"
namespace kern::recorder { bool StateMachine::process(Event) { return false; } }

// circular_log.hpp - public API already matches Day 4
```

```
#pragma once
#include "sensor_record.hpp"
#include <cstdint>
namespace kern::storage {
enum class StorageStatus : uint8_t { Ok, IoError, Corrupt, Full, BadMagic, NotMounted };
using RecordCb = bool(*)(const SensorRecord&, void*);
class CircularLog {
public:
    StorageStatus mount();
    StorageStatus writeRecord(const SensorRecord& r);
    StorageStatus replayNewest(uint32_t n, RecordCb cb, void* ctx);
    StorageStatus eraseAll(uint32_t magic);
    bool isMounted() const { return false; }
    uint32_t totalRecords() const { return 0; }
    uint32_t wrapCount() const { return 0; }
    uint8_t currentFile() const { return 0; }
    uint16_t writeIndex() const { return 0; }
};
}

// circular_log.cpp
#include "circular_log.hpp"
namespace kern::storage {
StorageStatus CircularLog::mount() { return StorageStatus::NotMounted; }
StorageStatus CircularLog::writeRecord(const SensorRecord&) { return StorageStatus::NotMounted; }
StorageStatus CircularLog::replayNewest(uint32_t, RecordCb, void*) { return StorageStatus::NotMounted; }
StorageStatus CircularLog::eraseAll(uint32_t) { return StorageStatus::NotMounted; }
}
```

recorder/sensor_bus.hpp

```
#pragma once
#include "../storage/sensor_record.hpp"
#include "FreeRTOS.h"
#include "semphr.h"
namespace kern::recorder {
class SensorBus {
public:
    void init() { m_mutex = xSemaphoreCreateMutexStatic(&m_mutexStorage); }
    void publish(const storage::SensorRecord& r) {
        if (m_mutex && xSemaphoreTake(m_mutex, pdMS_TO_TICKS(5)) == pdTRUE) {
            m_latest = r;
            xSemaphoreGive(m_mutex);
        }
    }
    storage::SensorRecord latest() {
        storage::SensorRecord r{};
        if (m_mutex && xSemaphoreTake(m_mutex, pdMS_TO_TICKS(5)) == pdTRUE) {
            r = m_latest;
            xSemaphoreGive(m_mutex);
        }
        return r;
    }
private:
    StaticSemaphore_t m_mutexStorage{};
    SemaphoreHandle_t m_mutex = nullptr;
    storage::SensorRecord m_latest{};
};
}
```

Minimal sensor and DSP interfaces

```
// lm35.hpp - photodiode.hpp and potentiometer.hpp follow the same pattern
#pragma once
namespace kern::sensors {
class Lm35 { public: void init() {} float readCelsius() { return 0.0f; } };
}

// dht11.hpp
#pragma once
namespace kern::sensors {
class Dht11 {
public:
    enum class Status { Ok, Timeout, CrcError };
    void init() {}
    Status read(float& tempC, float& humidity) { tempC=0; humidity=0; return Status::Timeout; }
};
}

// radiation_latch.hpp
#pragma once
namespace kern::sensors {
```

```
class RadiationLatch { public: void init() {} void isr() {} bool consumeEvent() { return false; } };
}

// buttons.hpp
#pragma once
namespace kern::sensors {
enum class PressType { None, Short, Long };
class Buttons { public: void init() {} PressType pollSw1() { return PressType::None; } };
}

// moving_average.hpp, threshold_detector.hpp, and channel.hpp
// may contain compile-only placeholders in Phase 0. Day 3 replaces them
// with the tested reference implementations and hysteresis behavior.
```

Communication stubs

```
// comm_link.hpp
#pragma once
#include "../protocol/frame.hpp"
namespace kern::recorder {
class CommLink {
public:
    void init() {}
    bool poll(protocol::Frame& out) { (void)out; return false; }
    void send(const protocol::Frame& f) { (void)f; }
};
}

// command_handler.hpp
#pragma once
#include "../protocol/frame.hpp"
namespace kern::recorder {
class CommandHandler {
public:
    void dispatch(const protocol::Frame& f) { (void)f; }
    void sendStatus() {}
};
}
```

0.12 Smoke-test orchestrator and task creation

```
// orchestrator.hpp
#pragma once
#include "../recorder/state_machine.hpp"
#include "../recorder/sensor_bus.hpp"
#include "../storage/circular_log.hpp"
#include "../recorder/comm_link.hpp"
#include "../recorder/command_handler.hpp"
namespace kern::system {
class Orchestrator {
public:
    void init();
    void runSensorTask();
    void runStorageTask();
    void runCommsTask();
    void runSystemTask();
    static Orchestrator& instance() { static Orchestrator o; return o; }
private:
    recorder::StateMachine sm;
    recorder::SensorBus bus;
    storage::CircularLog box;
    recorder::CommLink link;
    recorder::CommandHandler handler;
};
}

// orchestrator.cpp - Phase 0 smoke test
#include "orchestrator.hpp"
#include "../hal/gpio.hpp"
#include "../hal/watchdog.hpp"
#include "board.hpp"
#include "FreeRTOS.h"
#include "task.h"
#include <cstring>
extern UART_HandleTypeDef huart2;
extern IWDG_HandleTypeDef hiwdg;
namespace kern::system {
void Orchestrator::init() { bus.init(); }
void Orchestrator::runSensorTask()  { for (;;) vTaskDelay(pdMS_TO_TICKS(100)); }
void Orchestrator::runStorageTask() { for (;;) vTaskDelay(pdMS_TO_TICKS(100)); }
void Orchestrator::runCommsTask()   { for (;;) vTaskDelay(pdMS_TO_TICKS(10)); }
```

```
void Orchestrator::runSystemTask() {
    static const char msg[] = "KERN-LITE ALIVE\r\n";
    for (;;) {
        hal::gpio::toggle(board::LED1_BLUE);
        HAL_UART_Transmit(&huart2, reinterpret_cast<const uint8_t*>(msg), sizeof(msg)-1, 100);
        hal::watchdog::kick(hiwdg);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
}
```

```
// tasks.hpp
#pragma once
void kern_create_tasks();

// tasks.cpp
#include "tasks.hpp"
#include "tasks_defs.hpp"
#include "orchestrator.hpp"
#include "FreeRTOS.h"
#include "task.h"
using O = kern::system::Orchestrator;
static StaticTask_t sensorTcb, storageTcb, commsTcb, systemTcb;
static StackType_t sensorStack[kern::tasks::kSensorStack];
static StackType_t storageStack[kern::tasks::kStorageStack];
static StackType_t commsStack[kern::tasks::kCommsStack];
static StackType_t systemStack[kern::tasks::kSystemStack];
static void sensorTask(void*)  { O::instance().runSensorTask(); }
static void storageTask(void*) { O::instance().runStorageTask(); }
static void commsTask(void*)   { O::instance().runCommsTask(); }
static void systemTask(void*)  { O::instance().runSystemTask(); }
void kern_create_tasks() {
    O::instance().init();
    xTaskCreateStatic(sensorTask, "Sensor", kern::tasks::kSensorStack, nullptr,
                      kern::tasks::kSensorPrio, sensorStack, &sensorTcb);
    xTaskCreateStatic(storageTask, "Storage", kern::tasks::kStorageStack, nullptr,
                      kern::tasks::kStoragePrio, storageStack, &storageTcb);
    xTaskCreateStatic(commsTask, "Comms", kern::tasks::kCommsStack, nullptr,
                      kern::tasks::kCommsPrio, commsStack, &commsTcb);
    xTaskCreateStatic(systemTask, "System", kern::tasks::kSystemStack, nullptr,
                      kern::tasks::kSystemPrio, systemStack, &systemTcb);
}
extern "C" {
void vApplicationGetIdleTaskMemory(StaticTask_t** tcb, StackType_t** stack, uint32_t* size) {
    static StaticTask_t idleTcb;
    static StackType_t idleStack[configMINIMAL_STACK_SIZE];
    *tcb=&idleTcb; *stack=idleStack; *size=configMINIMAL_STACK_SIZE;
}
void vApplicationGetTimerTaskMemory(StaticTask_t** tcb, StackType_t** stack, uint32_t* size) {
    static StaticTask_t timerTcb;
    static StackType_t timerStack[configTIMER_TASK_STACK_DEPTH];
    *tcb=&timerTcb; *stack=timerStack; *size=configTIMER_TASK_STACK_DEPTH;
}
}

// init.hpp / init.cpp
#pragma once
extern "C" void kern_boot();

#include "init.hpp"
#include "tasks.hpp"
extern "C" void kern_boot() { kern_create_tasks(); }
```

SMOKE-TEST SCOPE The ASCII KERN-LITE ALIVE line is used only in Phase 0. Starting in Day 2, UART2 carries the binary framed protocol; periodic STATUS frames become the link heartbeat so raw text never corrupts protocol traffic.

0.13 Build, flash, and verify

- [ ] Build All. Fix every include path and compile error before adding functionality.
- [ ] Enable -Wall and -Wextra for project code. Treat warnings in student-owned code as errors.
- [ ] Flash through ST-Link and open the virtual COM port at 115200 8N1.
- [ ] Observe PC9 toggling once per second and KERN-LITE ALIVE once per second.
- [ ] Leave the board running for at least 30 seconds. It must not hard fault or repeatedly reset.
- [ ] Temporarily remove the watchdog refresh and confirm a reset occurs after approximately 4 seconds; then restore the refresh.
- [ ] Commit the .ioc, generated source, firmware scaffold, and README. Exclude Debug/, Release/, build artifacts, and editor files.

Phase 0 gate

- [ ] A clean clone builds with zero student-code warnings and zero errors.
- [ ] All four statically created tasks exist and the scheduler runs.
- [ ] PC9 heartbeat and UART2 smoke output are visible at 1 Hz.
- [ ] CMSIS-RTOS2 is selected in CubeMX.
- [ ] Default board initialization was generated, then every fixed pin was verified and conflicts were corrected.
- [ ] SPI initializes at 312.5 kHz, SD_CS starts HIGH, and the disk adapter has no fake fixed capacity.
- [ ] The canonical firmware/recorder paths and final protocol interfaces are present.
- [ ] Both team members can reproduce the result.

BLOCKER GATE Do not begin Day 1 until the complete Phase 0 demonstration passes on real hardware.

0.14 Common Phase 0 failures

CMSIS version mismatch: The middleware interface must be CMSIS_V2, not CMSIS_V1.

Wrong default-mode answer: Choose Yes when CubeIDE offers default peripheral modes, then override the generated assignments to match the fixed map.

PA5 conflict: The Nucleo default LED commonly uses PA5. KERN-LITE requires PA5 as SPI1_SCK.

SD initialization too fast: A prescaler of /64 at 80 MHz is 1.25 MHz and is not valid for SD initialization. Use /256.

Wrong watchdog reload: /256 with reload 1249 is about 10 seconds, not 4. Use reload 499 for the intended timeout.

Wrong EXTI IRQ: PB3 uses EXTI3_IRQn, not EXTI9_5_IRQn.

Renaming generated main.c: CubeMX may regenerate a second entry file. Keep generated C files and bridge into C++.

Dynamic mutex in a static design: Use xSemaphoreCreateMutexStatic for the SensorBus and TX mutex.

Raw text mixed with protocol: Disable Phase 0 ASCII output when binary framing begins.

Hard-coded SD capacity: FatFs must obtain the actual sector count from the card CSD.

DAY 1

PROTOCOL FOUNDATIONS

DAILY OBJECTIVE CRC-32 and the frame codec. The entire project depends on these being correct and bit-exactly agreed between firmware and Python. Nothing is wasted today - every module written is reused in every future day.

Member A - Firmware: CRC wrapper + frame codec

Morning (tasks A1.1 - A1.3)

A1.1 Create firmware/protocol/crc32.hpp and firmware/protocol/crc32.cpp.

Wrap an existing CRC-32 library (your vendor's HAL CRC peripheral configured for IEEE 802.3, or a software table). Expose exactly this API - do not change the signatures:

```
namespace kern::protocol {
    uint32_t crc32(const uint8_t* data, size_t len);
    uint32_t crc32Begin();
    uint32_t crc32Update(uint32_t crc, const uint8_t* data, size_t len);
    uint32_t crc32Finalize(uint32_t crc);
}
```

- Parameters that must match: polynomial 0xEDB88320 (reflected), init 0xFFFFFFFF, input reflected, output reflected, final XOR 0xFFFFFFFF. Write the software table version now; you can swap to the hardware peripheral later.

A1.2 Create firmware/protocol/frame.hpp. Define exactly:

- Constants: kStx = 0xAB, kEtx = 0xCD, kMaxPayload = 256, kFrameOverhead = 8.
- enum class FrameType : uint8_t with all opcodes from the spec (Section 7).
- enum class NackCode : uint8_t with all five codes.
- struct Frame { FrameType type; uint8_t payload[256]; uint16_t len; };

A1.3 Create firmware/protocol/codec.hpp and firmware/protocol/codec.cpp.

encode(const Frame& f, uint8_t* outBuf, size_t outSize) -> size_t - writes [STX][TYPE][LEN_LO][LEN_HI][PAYLOAD...][CRC0][CRC1][CRC2][CRC3][ETX], CRC over TYPE+LEN+PAYLOAD, returns total bytes written or 0 on overflow.

- class Decoder with states WaitStx -> Type -> LenLo -> LenHi -> Payload -> Crc0 -> Crc1 -> Crc2 -> Crc3 -> WaitEtx. Method feed(uint8_t byte) -> Result where Result ∈ {NeedMore, FrameReady, CrcError, SyncError}. Method const Frame& frame(). Method reset().
- CRC is accumulated incrementally as bytes are fed (TYPE, LEN_LO, LEN_HI, each payload byte), finalized in WaitEtx before comparison.

Afternoon (task A1.4)

A1.4 Write tests/host/test_codec.cpp. The file must compile and run standalone (no STM32 headers; include only protocol/crc32.* and protocol/codec.*). Implement these test cases as simple assert-based checks (or any unit test framework the team has used before):

CRC32("123456789") == 0xCBF43926 - the KAT. [BLOCKER] If this fails, stop. Fix the CRC parameters before anything else.

- Round-trip: encode a Status frame with 14-byte payload, feed all bytes to Decoder one at a time, verify FrameReady and decoded fields match.
- Round-trip: encode a zero-length payload ACK frame.
- Round-trip: encode a 256-byte payload frame (maximum size).
- Corruption: encode a valid frame, flip byte 5 of the payload, feed to decoder - must return CrcError, not FrameReady.
- Resync: feed 12 garbage bytes then a valid frame - decoder must reach FrameReady on the valid frame.
- Oversized LEN: craft bytes [0xAB][0x10][0xFF][0x01]... (LEN = 511 > 256) - decoder must return SyncError without overrunning the buffer.

Member B - Ground Station: CRC + frame codec in Python

Morning (tasks B1.1 - B1.2)

B1.1 Create groundstation/crc.py:

```
import zlib, struct
```

- def crc32(data: bytes) -> int:

  - return zlib.crc32(data) & 0xFFFFFFFF
- That one-liner is correct for IEEE 802.3 / Python zlib.crc32. Verify the KAT immediately: crc32(b"123456789") == 0xCBF43926. [BLOCKER] If this fails something is wrong with your Python environment.

B1.2 Create groundstation/frame.py. Define:

- All constants (STX, ETX, MAX_PAYLOAD, FRAME_OVERHEAD).
- FrameType and NackCode as IntEnum.
- @dataclass Frame: type: FrameType, payload: bytes, fields for decoded STATUS (state, sd_mounted, file_count, current_file, total_records, wrap_count, records_in_file).
- encode(frame: Frame) -> bytes - same layout as firmware encoder; CRC over TYPE+LEN+PAYLOAD.
- class Decoder: method feed(byte: int) -> Optional[Frame] returning a Frame when complete or None, raising CrcError or SyncError exceptions (or returning sentinel values - choose one convention and document it). Mirror the firmware state machine exactly.

Afternoon (task B1.3)

B1.3 Write tests/gs/test_codec.py (pytest):

- KAT for crc.crc32.
- Python encode -> Python decode round-trip for CMD_STATUS (no payload) and a crafted RECORD (32-byte payload).
- Corruption -> CrcError raised/returned.
- Resync after garbage.
- Cross-implementation vector: Take the raw bytes produced by frame.encode for a specific ACK frame and write them as a hex literal. Feed those exact bytes to the firmware decoder in test_codec.cpp. Confirm FrameReady with matching fields. Write the equivalent: take bytes from firmware encode (hardcode the output in a comment in test_codec.cpp) and decode them in Python. This is the most important test of the day.

Member C - (Absorb into A and B for 2-person teams)

Member C acts as reviewer and integration tester today:

Read both codecs and find any divergence in field order, endianness, or CRC coverage before EOD.

- Set up the test build system (a Makefile or CMakeLists.txt) that compiles tests/host/*.cpp against firmware/protocol/ with no STM32 headers. This build must work on a PC.
- Run both test suites and report results at check-in.

End-of-Day 1 Review (demonstration checklist)

- [ ] KAT: Run test_codec.cpp and test_codec.py. Demonstrate the output proving 0xCBF43926 on both.
- [ ] Cross-vector: Give me the exact byte sequence your Python encodes for an ACK frame (should be 8 bytes: AB 20 00 00 CRC0 CRC1 CRC2 CRC3 CD). Feed it to your firmware decoder. Does it return FrameReady?
- [ ] Corruption test: Confirm CrcError is returned (not FrameReady) when one payload byte is flipped.
- [ ] Resync test: Confirm decoder recovers after 12 garbage bytes and produces FrameReady on the following valid frame.
- [ ] Host build: Can I type one command and run all host tests? Demonstrate.

BLOCKER GATE The team does not begin Day 2 until tests 1 and 2 above pass. CRC mismatch at this stage poisons every subsequent day.

DAY 2

UART LINK AND COMMAND ROUND-TRIP

DAILY OBJECTIVE Move bytes across real UART and complete a CMD_STATUS -> STATUS -> ACK exchange. By end of day the GS can talk to the board.

Member A - Firmware: CommLink + Comms skeleton

Morning (tasks A2.1 - A2.2)

A2.1 Create firmware/recorder/comm_link.hpp and firmware/recorder/comm_link.cpp.

- CommLink wraps UART_HandleTypeDef*. Implement:
- init() - initializes a static mutex (xSemaphoreCreateMutexStatic) for TX, then arms byte-at-a-time RX interrupt (HAL_UART_Receive_IT).
- feed(uint8_t byte) - called from the UART RX ISR; feeds byte into the Decoder; sets m_frameReady = true and copies to m_pending when FrameReady is returned. Does not allocate.
- poll(Frame& out) - called from the Comms task; if m_frameReady, atomically copies m_pending to out, clears the flag, and returns true.
- send(const Frame& f) - takes the TX mutex, calls encode(f, buf, sizeof(buf)), transmits with HAL_UART_Transmit, releases the mutex.
- Define extern CommLink* g_commLink; and set it in init() so the UART ISR callback can call g_commLink->feed(byte).

A2.2 Create the RTOS task structure (stub versions only today): firmware/system/tasks_defs.hpp (stack sizes from the spec), firmware/system/tasks.cpp (four xTaskCreateStatic calls from boot()), and a stub Orchestrator in firmware/system/orchestrator.* that only holds CommLink and has a runCommsTask() body. The other three tasks can be infinite loops with vTaskDelay(100) for now.

- runCommsTask() body today:

```
void Orchestrator::runCommsTask() {
    for (;;) {
        kern::protocol::Frame f{};
        if (link.poll(f)) {
            handler.dispatch(f);   // stub handler today
        }
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}
```

Afternoon (task A2.3)

A2.3 Create a stub firmware/recorder/command_handler.*. Today implement only:

- sendAck() - sends a zero-payload ACK frame.
- sendNack(NackCode) - sends a 1-byte NACK frame.
- sendStatus() - sends a 14-byte STATUS frame: payload[0]=state(0), payload[1]=1 (SD mounted stub), payload[2]=4 (file_count), payload[3]=0 (current_file), payload[4..7]=0 (total_records), payload[8..11]=0 (wrap_count), payload[12..13]=0 (records_in_file).
- dispatch(Frame& f) - handles CMD_STATUS -> sendStatus(); any other type -> sendNack(BadCommand).
- Build and flash. Confirm the board is running (heartbeat LED blinking).

Member B - Ground Station: serial link manager

Morning (tasks B2.1 - B2.2)

B2.1 Create groundstation/link.py - SerialLink class:

- connect(port: str, baud: int = 115200) - open serial.Serial, set timeout=0.05.
- disconnect().
- send_frame(frame: Frame) - encode and write; increment TX counter; record timestamp for latency.
- receive_frame() -> Optional[Frame] - read available bytes, feed to Decoder one byte at a time; on FrameReady increment RX counter and compute latency if a pending command was issued; on CrcError increment CRC-error counter; on SyncError increment sync-error counter.
- Expose: rx_count, tx_count, crc_error_count, sync_error_count, last_latency_ms, rolling_avg_latency_ms, nack_count, nack_rate (NACK / commands sent).
- connection_state -> "connected" / "disconnected" / "reconnecting".
- Auto-reconnect thread: if connection drops, enter "reconnecting", retry every 2 s.

B2.2 Create groundstation/commands.py - CommandSender class:

- send_start(link) -> sends CMD_START (0x01, no payload).
- send_stop(link) -> sends CMD_STOP (0x02, no payload).
- send_status(link) -> sends CMD_STATUS (0x03, no payload).
- send_replay(link, n: int) -> sends CMD_REPLAY (0x04) with 2-byte LE n.
- send_erase(link, magic: int) -> sends CMD_ERASE (0x06) with 4-byte LE magic.
- All methods record the outgoing command type and timestamp on the link for latency tracking.

Afternoon (task B2.3)

B2.3 Write a small integration script groundstation/probe.py (not the final UI - just a script you run from the terminal):

```
python probe.py --port /dev/ttyACMx
```

It connects, sends CMD_STATUS, waits up to 2 s for a STATUS response, prints all 14 decoded fields, disconnects. Use this to verify the link. Also send an unknown frame type 0xFF and confirm the reply is NACK BadCommand.

- Write tests/gs/test_link.py:
- Feed a captured STATUS byte stream (hardcoded from the probe output) to the Decoder - verify all 14 fields decode correctly.
- Feed a NACK BadCommand byte stream - verify NackCode.BadCommand is decoded.

Member C

Run probe.py on the actual board; capture the raw byte stream with any serial monitor; paste the hex into test_link.py as the test vector.

- Verify TX mutex is working: write a small stress test that sends 50 CMD_STATUS frames as fast as possible and checks that all 50 responses come back with valid frame CRCs (no interleaving artifacts).

End-of-Day 2 Review

- [ ] Live check: Connect the GS to the board. Run probe.py. Demonstrate the decoded STATUS fields printed to the terminal.
- [ ] NACK check: Send opcode 0xFF. Demonstrate NACK BadCommand.
- [ ] Latency: What is last_latency_ms on a STATUS round-trip?
- [ ] Counters: After 10 status polls, what are rx_count and tx_count?
- [ ] Corruption resilience: Manually inject a bad byte into a frame (flip one byte in encode temporarily). Confirm crc_error_count increments and no crash.

BLOCKER GATE probe.py successfully exchanges CMD_STATUS -> STATUS on real hardware before Day 3 begins.

DAY 3

SENSORS, DSP, AND SENSOR RECORD ASSEMBLY

DAILY OBJECTIVE Produce a fully populated, CRC-protected SensorRecord at 10 Hz on firmware. In parallel the GS decodes records and builds the telemetry model. By end of day, live sensor values are visible on the GS.

TEAM SPLIT A works firmware sensors + DSP. B works GS telemetry decode + state display. These tracks are independent after the SensorRecord struct is agreed (it was locked on Day 1).

Member A - Firmware: sensor drivers, DSP pipeline, SensorRecord assembly

Morning (tasks A3.1 - A3.3)

A3.1 Implement or verify sensor drivers in firmware/sensors/:

lm35.hpp - already provided in the project; verify it compiles with your HAL and reads a plausible voltage from ADC_CHANNEL_9. Conversion: volts x 100.0f = degC.

- photodiode.hpp / potentiometer.hpp - read ADC_CHANNEL_6 and ADC_CHANNEL_5 respectively, return normalized float in [0.0, 1.0] via toVolts(raw) / 3.3f.
- dht11.* - timing-critical. DHT11 requires a GPIO toggle sequence with microsecond delays. Implement init() and read(float& tempC, float& humidity) -> Status. Use HAL_GetTick() for timeout detection. The bus pin is board::DHT_DATA (PB5). Return Status::Timeout or Status::CrcError on failure; do not block longer than 5 ms.
- radiation_latch.* - the trigger latch on an EXTI line. init() enables the interrupt; isr() is called from the EXTI ISR and increments m_count; consumeEvent() checks and clears via a semaphore. For now wire it to SW2 (PB3) as a manual trigger.
- buttons.* - already provided. Verify pollSw1() returns Short on a quick press and Long on a press held > 1 s.

A3.2 Copy firmware/dsp/moving_average.hpp, firmware/dsp/threshold_detector.hpp, and firmware/dsp/channel.hpp from the reference (they are already complete). Add firmware/system/config.hpp with all five ThresholdConfig constants and kDspWindow = 16.

A3.3 Implement runSensorTask() in firmware/system/orchestrator.cpp. The task loops every 100 ms (vTaskDelay(pdMS_TO_TICKS(100))). When not Recording (state machine not yet wired - stub isLogging() to return true for today), it still assembles and publishes records so you can observe them:

```
record.timestamp = HAL_GetTick() / 1000
record.ms = HAL_GetTick() % 1000
record.seq = ++m_recSeq
record.state = 0 (stub)
```

- lm35: read -> chLm35.process(tempC) -> rec.lm35_c = int16_t(out.filtered x 10)
- photo: read -> chPhoto.process(v) -> rec.light = uint16_t(out.filtered x 65535)
- pot: read -> chPot.process(v) -> rec.pot = uint16_t(out.filtered x 65535)
- dht11: read every 20 ticks (~2 s) -> rec.dht_temp_c, rec.dht_hum; on fault set fault_bits, keep last good
- latch: consumeEvent() -> no action today (event system removed)
- alert_bits: set each bit by comparing channel alert() to HighAlert / LowAlert
- fault_bits: set LM35_RANGE if tempC < -10 or > 100; set LIGHT_STUCK / POT_STUCK if value <= 0.001 or >= 0.999
- rec.crc32 = recordCrc(rec) (CRC-32 over bytes [0..27])
- bus.publish(rec)
- Also implement SensorBus (double-buffer + mutex) in firmware/recorder/sensor_bus.hpp.
- Also today: send each assembled record as a live RECORD frame from the Sensor task (temporary, until the Comms task owns streaming in Phase 5):
- cppFrame out{};
- out.type = FrameType::Record;
- out.len = sizeof(SensorRecord);
- memcpy(out.payload, &rec, sizeof(SensorRecord));
- link.send(out);

Afternoon (task A3.4)

A3.4 Write tests/host/test_dsp.cpp (no HAL headers - include only dsp/ headers):

Moving average: feed [0, 0, 0, 10, 10, 10] through MovingAverage<float, 3>, confirm the sequence of outputs is [0, 0, 0, 3.33, 6.67, 10.0] (within epsilon = 0.01).

- Threshold rising: start Normal, feed values until hi is crossed, confirm HighAlert; feed back to hi − hysteresis − epsilon, confirm Normal.
- Threshold falling: symmetric test for LowAlert.
- No chatter: feed a value exactly at hi − hysteresis repeatedly; confirm state does not toggle.
- static_assert(sizeof(SensorRecord) == 32) and static_assert(offsetof(SensorRecord, crc32) == 28).
- Record CRC: assemble a record with known fields (all zeros except seq=1), compute crc32(bytes 0..27), write the result into crc32 field, verify it matches.

Member B - Ground Station: record decode, telemetry model, state display

Morning (tasks B3.1 - B3.2)

B3.1 Create groundstation/telemetry.py - RecordDecoder and TelemetryModel classes.

RecordDecoder.decode(payload: bytes) -> SensorRecord - unpacks the 32-byte payload using struct.unpack_from('<IHHhhHHHBBB7xI', payload) (little-endian: u32 u16 u16 i16 i16 u16 u16 u16 u8 u8 u8 7xpadding u32). Returns a @dataclass SensorRecord with all fields plus computed properties:

- python@property
- def lm35_celsius(self): return self.lm35_c / 10.0
- @property
- def dht_temp_celsius(self): return self.dht_temp_c / 10.0
- @property
- def dht_humidity(self): return self.dht_hum / 10.0
- @property
- def light_normalized(self): return self.light / 65535.0
- @property
- def pot_normalized(self): return self.pot / 65535.0
- TelemetryModel holds a session list of SensorRecords and per-channel running stats. Add a method ingest(record: SensorRecord) that:
- Appends the record.
- Updates per-channel running min, max, mean (incremental - no recomputation from scratch).
- Updates alert activation counts and time-in-alert accumulators.

B3.2 Create groundstation/state.py - DeviceStateModel:

Stores current state (Idle / Recording / Fault), previous state, transition list (each entry: (wall_time, session_seq, from_state, to_state, duration_in_prev)).

- update_from_status(status_frame: Frame) - extracts payload[0], computes transition if state changed, logs it.
- state_name(code: int) -> str - {0: "Idle", 1: "Recording", 2: "Fault"}.
- command_allowed(cmd: FrameType) -> bool - state-gate table:
- CMD_START: only from Idle.
- CMD_STOP: only from Recording.
- CMD_REPLAY: only from Idle (and SD must be mounted from last STATUS).
- CMD_ERASE: only from Idle.
- CMD_STATUS: always.

Afternoon (task B3.3)

B3.3 Write tests/gs/test_telemetry.py:

Decode a hard-coded 32-byte record (craft one by hand: seq=5, lm35_c=253 meaning 25.3 degC, etc.) and verify all scaled properties.

- Record-CRC check: compute crc32(payload[0:28]) and compare to payload[28:32] (little-endian u32). Verify it matches for your hand-crafted record (compute the expected CRC in Python and write it into the test vector).
- TelemetryModel: ingest 5 known records, verify min/max/mean correct.
- DeviceStateModel: simulate STATUS updates 0 -> 1 -> 0 -> 2, verify transition list length == 3 and durations are non-negative.
- Command gating: in Recording state, CMD_START is not allowed, CMD_STOP is allowed.

Member C

Read test_dsp.cpp and test_telemetry.py, run them, report any failures.

- Connect the board with today's firmware to the GS. Write a 30-second capture using probe.py extended to also print each RECORD frame's decoded fields as it arrives. Confirm LM35 reads a plausible temperature, light and pot read mid-scale, DHT11 is reasonable.
- Verify seq is strictly monotonic across all received records and no CRC failures appear in that 30-second window.

End-of-Day 3 Review

- [ ] Live sensor values: Open the GS and demonstrate five consecutive RECORD frames with decoded values. Are LM35, DHT11 temp/humidity, light, and pot all in a plausible range?

- [ ] Hysteresis: Cover the photodiode with your hand (drive light -> 0) and uncover it. Does LIGHT_LO alert bit set and clear? Does it clear only after light returns well above the threshold, not immediately at the boundary?
- [ ] DHT11 fault: Disconnect the DHT11 data pin momentarily. Does fault_bits show DHT_TIMEOUT? Does the last-known-good temperature still appear in the record (not zero)?
- [ ] seq: Demonstrate 10 consecutive records. Is seq monotonically increasing with no gaps?
- [ ] Record CRC: Print crc32(payload[0:28]) in Python for one received record. Does it match payload[28:32]?

BLOCKER GATE Record CRC must validate on every received record before Day 4 begins.

DAY 4

CIRCULAR FILE STORAGE

DAILY OBJECTIVE The new core of this project. Build CircularLog - multi-file ring, sequential write, file advance, wraparound, metadata persistence, crash recovery, and REPLAY. By end of day you can fill and wrap the ring and watch it on the GS storage panel.

TEAM SPLIT A + C on firmware storage (the most complex firmware task of the week). B on the GS storage panel and session management foundation.

Member A - Firmware: CircularLog implementation

Full day (tasks A4.1 - A4.4)

A4.1 Create firmware/storage/circular_log.hpp. Define:

```
namespace kern::storage {

inline constexpr uint8_t LOG_FILE_COUNT = 4;
inline constexpr uint16_t RECORDS_PER_FILE = 256;
inline constexpr uint16_t META_FLUSH_EVERY_N = 16;
inline constexpr uint32_t ERASE_MAGIC = 0xDEADC0DEu;
inline constexpr uint32_t META_MAGIC = 0x4C4F4700u; // "LOG\0"
inline constexpr uint32_t META_VERSION = 1u;

struct LogMeta { // stored in META.BIN, #pragma pack(1)
    uint32_t magic;
    uint32_t version;
    uint8_t  file_count;
    uint16_t records_per_file;
    uint8_t  current_file;
    uint16_t write_index;
    uint32_t wrap_count;
    uint32_t total_records;
    uint8_t  reserved[10];      // pad to 32 bytes before CRC
    uint32_t crc32;             // over all preceding bytes
};
static_assert(sizeof(LogMeta) == 36, "LogMeta size");

enum class StorageStatus : uint8_t {
    Ok, IoError, Corrupt, Full, BadMagic, NotMounted
};

using RecordCb = bool(*)(const SensorRecord&, void* ctx);

class CircularLog {
public:
    StorageStatus mount();
    StorageStatus writeRecord(const SensorRecord& r);
    StorageStatus replayNewest(uint32_t n, RecordCb cb, void* ctx);
    StorageStatus eraseAll(uint32_t magic);

    uint32_t totalRecords()  const { return m_meta.total_records; }
    uint32_t wrapCount()     const { return m_meta.wrap_count; }
    uint8_t  currentFile()   const { return m_meta.current_file; }
    uint16_t writeIndex()    const { return m_meta.write_index; }
    bool     isMounted()     const { return m_mounted; }

private:
    StorageStatus readMeta();
    StorageStatus writeMeta();
    StorageStatus recoverPosition();
    uint32_t      metaCrc(const LogMeta& m);
    uint32_t      recordCrc(const SensorRecord& r);

    FATFS  m_fatfs{};
    FIL    m_files[LOG_FILE_COUNT]{};
    bool   m_filesOpen[LOG_FILE_COUNT]{};
    LogMeta m_meta{};
    bool   m_mounted = false;
};
} // namespace kern::storage
```

A4.2 Implement firmware/storage/circular_log.cpp. Implement each method:

- mount():
- f_mount(&m_fatfs, "0:", 1).
  - Open/create LOG00.BIN ... LOG03.BIN with FA_READ | FA_WRITE | FA_OPEN_ALWAYS.
  - Call readMeta(). If metadata is valid, scan forward from the saved head to recover records written since the last metadata flush. If metadata is Corrupt or unreadable, call recoverPosition() for a full scan.
  - If no valid record found at all, zero-initialize m_meta and call writeMeta().
  - Set m_mounted = true.
- readMeta(): open META.BIN, read 36 bytes into LogMeta, verify magic, version, and crc32. Return Corrupt if any check fails.
- writeMeta(): recompute CRC, write 36 bytes to META.BIN, f_sync.
- recoverPosition(): scan every slot, validate record CRCs, and determine the newest valid record using timestamp, ms, and sequence ordering. The next circular slot is the write head. A simple valid-to-invalid boundary is not sufficient after a complete wrap because every slot may contain a valid older record.
- writeRecord(r):
- Create SensorRecord stored = r; then set stored.crc32 = recordCrc(stored). Do not modify the const input reference.
  - Seek to write_index x 32 in m_files[current_file] and write stored.
  - Write the 32-byte stored record.
  - Increment write_index.
  - If write_index == RECORDS_PER_FILE: set write_index = 0, current_file = (current_file + 1) % LOG_FILE_COUNT. If we just wrapped back to 0 from LOG_FILE_COUNT−1, increment wrap_count.
  - Increment total_records.
  - If total_records % META_FLUSH_EVERY_N == 0, call writeMeta().
- replayNewest(n, cb, ctx):
  - Clamp n to min(n, min(total_records, total_capacity)). Only records still retained in the ring are replayable.
  - Compute global write position G = current_file x RECORDS_PER_FILE + write_index.
  - Start position = (G - n + total_capacity) % total_capacity where total_capacity = LOG_FILE_COUNT x RECORDS_PER_FILE.
  - Walk forward n steps: compute file and record index, read the stored record, and call cb in chronological order. If the record-level CRC is invalid, still deliver the raw stored record so the ground station can report storage corruption; remember the error and continue with surrounding records. Stop only if cb returns false or the file read itself fails.
- eraseAll(magic): verify magic == ERASE_MAGIC else return BadMagic; close files; f_unlink each log file + META.BIN; call mount() to reinitialize.

A4.3 Update runStorageTask() stub in orchestrator.cpp to call box.writeRecord(bus.latest()) every time the Sensor task posts a record. Do not wire state guards yet (that is Day 5); just write if mounted.

A4.4 Write tests/host/test_storage.cpp - pure index math, no FatFs calls (mock or stub FatFs with a thin shim writing to regular FILE*). Tests:

Write exactly RECORDS_PER_FILE records to the ring; confirm current_file advances from 0 to 1 and write_index resets to 0.

  - Write LOG_FILE_COUNT x RECORDS_PER_FILE records; confirm wrap_count == 1 and current_file == 0, write_index == 0.
  - Write one more record after a full wrap; confirm the oldest record in file 0 position 0 is overwritten.
- replayNewest(10) after 15 records written (no wrap): returns exactly 10 records in chronological order, all CRC-valid.
- replayNewest(400) after 300 records (no wrap): clamps to 300, returns all 300.
- replayNewest(20) that crosses a file boundary (e.g. current_file=1, write_index=5, N=20): returns records from file 0 positions 241-255 then file 1 positions 0-4.
- eraseAll(0x00000000) -> BadMagic, state unchanged.
- Meta CRC: construct a valid LogMeta, corrupt one byte, readMeta returns Corrupt.
- Recovery: corrupt meta, mark record 37 in file 0 as the last CRC-valid record (all later bytes garbage), confirm recoverPosition() lands on current_file=0, write_index=38.

Member B - Ground Station: storage panel + session management

Full day (tasks B4.1 - B4.2)

B4.1 Create groundstation/storage_panel.py - StorageModel:

update_from_status(frame: Frame) - decode all 14 STATUS bytes: state, sd_mounted, file_count, current_file, total_records, wrap_count, records_in_file.

- Expose properties for the UI: ring_visual() -> list of dicts, one per file: {index, is_current, is_wrapped, record_count} (before first wrap: only files 0..current_file have records; after first wrap, all files are full except current).
- Track live_record_count (records received as live RECORD frames) and replay_record_count (records received after a REPLAY command) as separate counters incremented by the GS when records arrive.

B4.2 Create groundstation/session.py - Session:

on_connect(port: str) - create a new timestamped session directory: sessions/YYYY-MM-DD_HH-MM-SS/.

- append_record(record: SensorRecord, wall_time: float) - append 32 binary record bytes + 8-byte wall-clock timestamp (float64 LE) to session.bin; auto-flush every 16 records.
- append_raw_frame(direction: str, frame_bytes: bytes, wall_time: float) - append a hex+timestamp line to frames.log.

- close() - flush and close all files.
- load(path: str) -> Session - reconstruct a session from session.bin (for reopening previous sessions).
  - Write tests/gs/test_session.py:
  - Write 10 records to a session, close it, reload it, confirm all 10 records decode correctly.
  - Confirm wall-clock timestamps are monotonically increasing.

Member C

Run test_storage.cpp host tests, report results.

- On the board: after Day 4 firmware is flashed, let it run until it wraps once (~100 s at 10 Hz). Inspect META.BIN and all four LOG*.BIN files using a hex dump tool - manually verify file sizes are 256x32 = 8192 bytes each, current_file in META is correct, and wrap_count == 1.
- Power-cycle the board mid-write (while it is recording) and confirm it continues writing without corrupting the records that came before the reset (verify by checking seq continuity on the GS).

End-of-Day 4 Review

- [ ] File ring: Let the board run for 110 seconds. Open the GS and demonstrate the storage panel reporting wrap_count = 1.
- [ ] File advance: Demonstrate the storage panel as current_file goes from 0 -> 1 -> 2 -> 3 -> 0.
- [ ] REPLAY across boundary: Issue CMD_REPLAY 300 after a wrap. Demonstrate that you receive records from two different files, all with valid record CRCs.
- [ ] Recovery: Power-cycle mid-write. After reboot, demonstrate seq continues from where it left off (no gap larger than a few records) and the newest pre-reset records are still intact (verify with CMD_REPLAY).
- [ ] Storage tests: Run tests/host/test_storage.cpp. Demonstrate all tests green.

BLOCKER GATE REPLAY across a file boundary must work and recovery must restore the correct position before Day 5.

DAY 5

STATE MACHINE, FULL COMMAND HANDLER, RTOS WIRING

DAILY OBJECTIVE Wire the firmware FSM into the RTOS, implement all commands with proper state guards, and start live streaming. By end of day the GS can START -> watch live data -> STOP -> REPLAY -> ERASE.

Member A - Firmware: state machine + full orchestrator

Morning (tasks A5.1 - A5.2)

A5.1 Create firmware/recorder/state_machine.*. Implement the three-state FSM exactly:

- States: Idle (0), Recording (1), Fault (2)
- Events: ChecksPassed, SdFault, UartStart, UartStop, ShortPress, FaultCleared
- Transitions:
- Idle + ChecksPassed -> Idle (startup checks complete; wait for START)
- Idle + UartStart -> Recording
- Idle + SdFault -> Fault (the orchestrator emits this event after the third consecutive failure)
- Recording + UartStop -> Idle (flush meta)
- Recording + ShortPress -> Idle (flush meta)
- Recording + SdFault -> Fault
- Fault + FaultCleared -> Recording (resume after a successful remount)
- Fault: orchestrator retries mount; after 3 fails -> reboot
- onEnter(Recording): tone chirp, green LED
- onEnter(Idle): silence, LED off
- onEnter(Fault): tone fault, red LED blink

A5.2 Complete firmware/recorder/command_handler.cpp. Implement all five commands with guards:

CMD_START (0x01): if not Idle -> NACK InvalidState; else sm.process(UartStart), sendStatus(), sendAck().

- CMD_STOP (0x02): if not Recording -> NACK InvalidState; else sm.process(UartStop), box.writeMeta() (flush), sendStatus(), sendAck().
- CMD_STATUS (0x03): sendStatus() (always).
- CMD_REPLAY (0x04): if not Idle -> NACK InvalidState; if not mounted -> NACK StorageError; read u16 n from payload (default 120 if len < 2); call box.replayNewest(n, ...) streaming each record as a RECORD frame; sendAck().
- CMD_ERASE (0x06): if not Idle -> NACK InvalidState; if len < 4 -> NACK BadMagic; read u32 magic; box.eraseAll(magic) -> on fail NACK BadMagic, on success sendAck().
- Default: NACK BadCommand.
- sendStatus() must now use real values from sm.state(), box.isMounted(), LOG_FILE_COUNT, box.currentFile(), box.totalRecords(), box.wrapCount(), box.writeIndex().

Afternoon (tasks A5.3 - A5.4)

A5.3 Complete firmware/system/orchestrator.cpp - all four tasks:

runSensorTask(): only sample and publish when sm.isLogging() (i.e. Recording); delay and skip otherwise. Stream each record live as a RECORD frame through link.send().

- runStorageTask(): only write when sm.isLogging(); SD-write fail policy: tolerate MAX_WRITE_FAILS=3 consecutive failures, then sm.process(SdFault). Yield with vTaskDelay(pdMS_TO_TICKS(100)).
- runCommsTask(): link.poll(f) -> handler.dispatch(f), every 10 ms.
- runSystemTask(): every 50 ms: kick watchdog; drive LEDs per state; poll buttons.pollSw1() -> short press -> sm.process(ShortPress) -> sendStatus(); every 5 s send a framed STATUS as the protocol heartbeat. Do not print raw ASCII on the protocol UART after Day 2.

A5.4 Write tests/host/test_fsm.cpp:

- Each valid (state, event) pair -> correct next state.
- CMD_START from Recording -> NACK InvalidState.
- CMD_STOP from Idle -> NACK InvalidState.
- CMD_ERASE from Recording -> NACK InvalidState.
- CMD_ERASE wrong magic -> NACK BadMagic.
- CMD_REPLAY from Fault -> NACK InvalidState.
- Write-failure policy: the orchestrator remains Recording after the first two consecutive write failures; the third failure emits one SdFault event and the FSM transitions to Fault.
- FaultCleared from Fault -> Recording.

Member B - Ground Station: full command panel + REPLAY display + integrity

Morning (tasks B5.1 - B5.2)

B5.1 Update groundstation/commands.py - add auto-poll:

StatusPoller thread: sends CMD_STATUS every poll_interval_s (default 5); updates DeviceStateModel on reply.

- On receiving a STATUS, detect reboot: if total_records in new STATUS is far less than the last known value, flag a "REBOOT_DETECTED" event, log it, and re-issue STATUS.

B5.2 Create groundstation/integrity.py - IntegrityChecker:

check_frame(frame: Frame) -> bool - frame CRC is already validated by the decoder; this method re-validates the record-level CRC for RECORD frames: crc32(payload[0:28]) == struct.unpack_from('<I', payload, 28)[0]. If frame CRC passes but record CRC fails -> log "STORAGE_CORRUPTION_WARNING", return False.

- check_sequence(record: SensorRecord, last_seq: int) -> int - returns gap size; if record.seq != (last_seq + 1) % 65536: log "SEQ_GAP size=N".

Afternoon (task B5.3)

B5.3 Wire the full receive path in groundstation/link.py:

When a RECORD frame arrives:

- IntegrityChecker.check_frame(frame).
- TelemetryModel.ingest(record).
- Session.append_record(record, time.time()).
  - Increment live or replay counter on StorageModel.

When a STATUS frame arrives:

- DeviceStateModel.update_from_status(frame).
- StorageModel.update_from_status(frame).

When a NACK arrives:

- Decode NackCode, log to alert history, display in command panel.
  - Write tests/gs/test_integrity.py:
- A record with valid frame + valid record CRC -> no warning.
- A record with valid frame but corrupted byte 10 of payload -> STORAGE_CORRUPTION_WARNING.
- Sequence: records with seq = 1, 2, 3, 7 -> gap of 3 logged at record 7.

Member C

Run all three test suites (test_codec, test_dsp, test_storage, test_fsm on host; test_codec, test_link, test_telemetry, test_integrity in pytest). Report any failures immediately.

- Live integration test: run the full GS against the Day-5 firmware. Perform the sequence: START -> observe live streaming for 30 s -> STOP -> REPLAY 60 -> observe 60 records arrive -> ERASE -> confirm new STATUS shows total_records = 0. Report any discrepancies.

End-of-Day 5 Review

- [ ] START -> Recording: Send CMD_START. Demonstrate STATUS replies state=1. Demonstrate live RECORD frames arriving in the GS.
- [ ] STOP -> Idle: Send CMD_STOP. Confirm streaming halts and STATUS replies state=0. Confirm META.BIN was updated (inspect with hex dump).
- [ ] REPLAY: After STOP, send CMD_REPLAY 60. Demonstrate 60 records arrive, all passing both CRC checks.
- [ ] Bad command guards: From Recording, send CMD_START again -> demonstrate NACK InvalidState. From Idle, send CMD_STOP -> NACK InvalidState.
- [ ] ERASE: From Idle, send CMD_ERASE with magic 0xDEADC0DE. Confirm ACK and next STATUS shows total_records=0, wrap_count=0.
- [ ] FSM tests: Demonstrate tests/host/test_fsm.cpp passing.

BLOCKER GATE The full five-command cycle works on hardware before Day 6.

DAY 6

GROUND STATION ANALYTICS

DAILY OBJECTIVE Complete the full GS: rolling chart, per-channel statistics, state-transition timeline, alert/event history log, link-quality indicator, reconnect, reboot detection, and exports. This is the heaviest Python day.

TEAM SPLIT B continues leading GS. A uses this day for firmware hardening and the framed STATUS heartbeat. C leads the analytics tests.

Member A - Firmware hardening

Morning (task A6.1)

A6.1 Audit and harden the firmware against all fault scenarios that end-of-week testing will exercise:

Watchdog: confirm runSystemTask kicks IWDG every cycle. Confirm no other task can starve it for >3 s.

- SD write-fail resilience: inject consecutive write failures. The first two failures keep the system in Recording; the third enters Fault. Restore storage, remount successfully, emit FaultCleared, and verify the system resumes Recording.
- Metadata flush on STOP: confirm writeMeta() is called in CMD_STOP handler. Power-cycle immediately after STOP and verify metadata matches the last record written.
- Heartbeat: confirm a valid framed STATUS is transmitted every 5 s in both Idle and Recording states; no raw text may appear on the protocol UART.
- LED states: solid green = Recording+nominal; blinking green = Recording+fault_bits set; off = Idle; blinking red = Fault.
- Button: short press from Recording -> Idle; verify metadata is flushed.

Afternoon (task A6.2)

A6.2 Write the design note draft in docs/design_note.md:

- Your LogMeta byte layout (table of fields + offsets).
- Your recovery algorithm in prose (3-5 sentences).
- One robustness scenario you tested (pulled power mid-write): what you expected, what you observed, result.

Member B - Ground Station: chart, stats, timeline, history, quality, export

Morning (tasks B6.1 - B6.3)

B6.1 Create groundstation/chart.py - RollingChart:

- Holds a deque of the last N records (default 120).
- update(record) - append, drop oldest if at capacity.
- state_marker(state_change) - records a vertical marker position.
- reboot_marker(seq) - marks a reboot event.
- gap_notch(seq, gap_size) - marks a sequence gap.
- Expose per-channel arrays for plotting: raw values, alert-shading booleans, threshold lo/hi lines.
- render(ax_dict) - draws all five channels onto provided matplotlib Axes with threshold lines, alert shading, and markers. Each channel is toggle-able.

B6.2 Create groundstation/stats.py - ChannelStats:

- For each of the five channels maintain incrementally (no recomputation):
- n, min_val (with seq), max_val (with seq), mean, M2 (Welford's online algorithm for variance -> stddev = sqrt(M2 / (n-1))).
- alert_activations: incremented each time the channel transitions into alert.
- time_in_alert_s: accumulated by comparing consecutive record timestamps.
- pct_in_alert: time_in_alert_s / session_duration_s x 100.

B6.3 Create groundstation/timeline.py - StateTimeline:

- on_state_change(from_state, to_state, wall_time, session_record_count) - append a segment.
- Each segment: {state, start_wall, end_wall, duration_s, start_seq, end_seq, record_count}.
- on_alert(channel, active: bool, seq, wall_time) - append to alert track.
- on_reboot(seq, wall_time).
- export_text(path) - write a human-readable text file: one line per state band with wall-clock timestamps, durations, and record counts; alert track below; reboot events.

Afternoon (tasks B6.4 - B6.5)

B6.4 Create groundstation/alert_log.py - AlertLog:

One list of LogEntry objects, each with (category, wall_time, session_seq, message). Categories: ALERT_ACTIVE, ALERT_CLEAR, STATE_TRANSITION, NACK, CRC_ERROR, SYNC_ERROR, SEQ_GAP, REBOOT, HEARTBEAT_TIMEOUT, GS_EVENT. Methods: add(category, ...), filter(category_set), sort_by_time(), export_text(path).

B6.5 Create groundstation/link_quality.py - LinkQualityMonitor:

- Compute a 0-100% score every 5 s from:

- CRC error rate (weight 30%).
- Sync error rate (weight 20%).
- Sequence-gap rate (weight 25%).
- NACK rate (weight 15%).
- Frame timing jitter (stddev of inter-frame interval, weight 10%).
- Score = 100 x (1 − weighted_error_rate), clamped [0, 100]. Expose quality_pct, degraded (< 70), poor (< 40).
- Also add a heartbeat monitor: if no valid STATUS or RECORD frame is seen for 15 s while connected, log HEARTBEAT_TIMEOUT.
- Update groundstation/export.py:
- session_to_csv(session, path) - all records, all fields scaled, human-readable headers.
- alert_log_to_text(alert_log, path).
- raw_frame_log_to_text(session, path).
- timeline_to_text(timeline, path).

Member C - Analytics tests

Full day (tasks C6.1 - C6.3)

C6.1 Write tests/gs/test_stats.py:

Feed [10.0, 20.0, 30.0, 40.0, 50.0] to ChannelStats.update(). Verify mean == 30.0, stddev ~ 15.81, min == 10, max == 50.

- Compare incremental stddev to numpy.std([...], ddof=1) on the same series (within 1e-6).
- Alert timing: records at t=0, 1, 2, 3, 4 seconds; alert active at t=1, 2, 3; verify time_in_alert_s == 2.

C6.2 Write tests/gs/test_timeline.py:

Simulate state changes Idle -> Recording (t=0, seq=0), Recording -> Idle (t=30, seq=300), Idle -> Recording (t=60, seq=300).

- Verify export text contains two Recording bands of 30 s each.
- Verify a reboot event appears at the correct position.

C6.3 Write tests/gs/test_link_quality.py:

- Zero errors, zero jitter -> quality = 100%.
- CRC rate = 10%, sync rate = 5%, gap rate = 5%, NACK rate = 0%, jitter = low -> quality < 80%.
- Reboot detector: inject a status where total_records drops from 1000 to 3 -> REBOOT event fired.

End-of-Day 6 Review

- [ ] Chart: Start a recording session. Demonstrate the rolling chart updating live with all five channels, threshold lines visible, and alert shading when you move the potentiometer to an extreme.
- [ ] Stats: After 60 s of recording, demonstrate per-channel min/max/mean/stddev. Cross-check lm35_celsius mean against a simple sum/n in your head.
- [ ] State timeline: Perform Idle -> Recording (30 s) -> Idle. Demonstrate the timeline export text file.
- [ ] Alert log: Trigger a light alert (cover photodiode). Demonstrate the alert log entry with its seq and timestamp.
- [ ] Link quality: What is the link-quality score on a clean link? Does it decrease when you deliberately inject a CRC error (flip a byte in encode, send once, revert)?
- [ ] Analytics tests: Run test_stats.py, test_timeline.py, test_link_quality.py. Demonstrate all green.

BLOCKER GATE All GS panels must update correctly from live data before Day 7.

DAY 7

INTEGRATION, VALIDATION, AND DEMO

DAILY OBJECTIVE Stress-test the full system, inject every fault category, produce the demonstration artifacts, and prepare the design note and README. This is the final quality gate - every scenario from the spec is exercised today.

Morning - Fault injection battery (whole team, 3 hours)

Execute these test scenarios in order. Record pass/fail in a table in docs/.

T1 Full wraparound demo

Run the board for 110 s from a clean erase (START -> let it fill all 4 files -> wrap at least once -> STOP). The GS storage panel must show wrap_count >= 1, current_file cycling, records_in_file resetting. Capture a screenshot.

T2 REPLAY across file boundary

After T1, issue CMD_REPLAY 300. Verify: exactly 300 records arrive; all pass both CRC checks; the records span at least two different log files (check by seq range); the last record's seq matches the last recorded seq before STOP.

T3 REPLAY overflow

Issue CMD_REPLAY 65535. Verify: exactly min(65535, min(total_records, ring_capacity)) records arrive; every uncorrupted record validates; ACK follows.

T4 REPLAY with N = 0

- Issue CMD_REPLAY 0. Verify: zero records arrive; ACK follows immediately; no crash.

T5 Power-loss mid-write recovery

START -> let run for 40 s -> pull power abruptly -> restore power -> let boot -> connect GS -> verify:

- REBOOT_DETECTED fires in GS.
- total_records in STATUS is close to (but may be up to META_FLUSH_EVERY_N less than) the pre-reset value.
  - Issue CMD_REPLAY 20. Verify 20 valid records arrive with valid record CRCs.
- STOP, then START again. Verify new records write at the correct position (no overwriting valid records from before the reset; no gap larger than META_FLUSH_EVERY_N records).

T6 Corrupt frame on the link

Run two checks: send a malformed GS-to-board frame and verify the firmware ignores it without crashing; then inject one corrupted board-to-GS byte stream and verify the GS crc_error_count increments. Streaming must continue after both checks.

T7 Corrupt stored record

After a recording session, use f_lseek + f_write on a PC-mounted SD to corrupt bytes 10-13 of record 5 in LOG01.BIN. Re-insert the SD; connect GS; issue CMD_REPLAY covering that record. Verify: GS logs STORAGE_CORRUPTION_WARNING for that record; surrounding records still deliver correctly.

T8 ERASE wrong magic

From Idle, send CMD_ERASE with magic 0x00000000. Verify: NACK BadMagic; files are untouched (total_records unchanged in next STATUS).

T9 Command gating

- From Recording: CMD_START -> NACK InvalidState.
- From Recording: CMD_ERASE 0xDEADC0DE -> NACK InvalidState.
- From Idle: CMD_STOP -> NACK InvalidState.
- From Fault: CMD_REPLAY -> NACK InvalidState.

T10 Link drop / auto-reconnect

While streaming, unplug and re-plug the USB-UART cable. Verify: GS enters "reconnecting" state; on reconnect, issues CMD_STATUS; streaming resumes; rx_count and tx_count continue (don't reset).

T11 DHT11 fault recovery

Disconnect the DHT11 data pin mid-recording. Verify: fault_bits & 0x02 (DHT_TIMEOUT) set on records; temperature and humidity hold last-known-good (not zero); recording continues; no FSM fault transition.

T12 SD fault and recovery (if injectable)

If your board allows SD hotplug: remove SD mid-recording. Verify: after MAX_WRITE_FAILS=3 failed writes the FSM enters Fault; LED blinks red; GS receives STATUS with state=2. Re-insert SD. Verify: firmware remounts, returns to Recording, sends STATUS state=1.

Afternoon - Demo artifacts and documentation (whole team, 3 hours)

D1 Export the demo session (Member B)

- From T1 + T2:
- Export session CSV. Open in a spreadsheet; verify headers, all five channels, scaled values.
- Export alert log text.
- Export state timeline text.
- Export raw frame log.
- Save all four to docs/demo/.

D2 Screenshot (Member C)

Capture a screenshot of the GS mid-wraparound (storage panel showing file advance, chart scrolling, stats panel populated). Save to docs/demo/screenshot_wraparound.png.

D3 README (Member A)

- Write docs/README.md with:
- Build and flash steps (exact commands for your toolchain).
- GS run steps (pip install -r requirements.txt, python groundstation/main.py --port /dev/ttyACMx).
- Wiring / pinout table (UART2 TX/RX, SD card SPI pins, sensor connections).
- CRC known-answer: CRC32("123456789") = 0xCBF43926 - confirmed on firmware and Python.
- Configured constants: LOG_FILE_COUNT=4, RECORDS_PER_FILE=256, META_FLUSH_EVERY_N=16, sample rate 10 Hz.
- How to run all unit tests (one command each for host C++ tests and pytest).

D4 Design note (Member A, finalize)

- Complete docs/design_note.md:
- LogMeta byte layout table.
- Recovery algorithm.
- One robustness scenario (from T5).

D5 Run full test suite one last time (Member C)

- Run every test:

```
# Host C++ tests
cd tests/host && make && ./test_codec && ./test_dsp && ./test_storage && ./test_fsm
# Python tests
cd tests/gs && pytest -v
All must pass. Fix anything that broke during Day 7 changes.
```

End-of-Day 7 Review (Final Demo)

- [ ] This is the instructor demo. Present each item in order:
- [ ] T1 demo live: Start fresh (erase), record for 110 s, show wrap_count increment in real time.
- [ ] T2: Issue REPLAY 300. Show 300 records arrive, all CRC-green, spanning two files.
- [ ] T5 evidence: Show the design note + your T5 test procedure + result. Pull power mid-write in front of me and show recovery.
- [ ] T6 + T7: Inject one frame-CRC error (show counter increment). Show the storage-corruption warning from T7.
- [ ] T9: Run the command-gating checks from Idle and Recording states.
- [ ] Full test suite: Run make && ./test_codec && ./test_dsp && ./test_storage && ./test_fsm and pytest -v live. Show all green.
- [ ] CSV: Open the exported CSV in a spreadsheet. Demonstrate the lm35_celsius column with plausible values.
- [ ] Screenshot + design note: Show both documents in docs/.

PROGRESS TRACKER AND FINAL CHECKS

Use this page during phase reviews. Record whether the gate passed and identify the exact blocker that must be removed before the team continues.

Phase	Theme	Gate evidence	Status	Blocker / next action
0	Setup	Clean build; CMSIS-RTOS2; scheduler, LED, UART, watchdog smoke test	[ ] Pass [ ] Fail	
1	CRC + codec	KAT 0xCBF43926 on both ends; cross-vector round-trip	[ ] Pass [ ] Fail	
2	UART link	probe.py receives STATUS from real hardware	[ ] Pass [ ] Fail	
3	Sensors + DSP	Record CRC validates on every received record	[ ] Pass [ ] Fail	
4	Circular files	REPLAY crosses a file boundary; recovery works after power-cycle	[ ] Pass [ ] Fail	
5	FSM + commands	Full five-command cycle on hardware; FSM tests pass	[ ] Pass [ ] Fail	
6	GS analytics	All GS panels update from live data; analytics tests pass	[ ] Pass [ ] Fail	
7	Integration	T1-T12 battery passes; full test suite and artifacts complete	[ ] Pass [ ] Fail	

System-wide correctness rules

One UART, one wire format: Phase 0 may print ASCII. From Day 2 onward USART2 carries only framed binary data; STATUS frames are the heartbeat.

CRC byte order: Transmit CRC-32 little-endian and use the same coverage on firmware and Python.

Retained-record count: The ring can replay at most LOG_FILE_COUNT x RECORDS_PER_FILE records, even if lifetime total_records is larger.

Crash recovery: Valid metadata may be stale. Recovery must scan forward from the saved head before accepting it.

Storage corruption visibility: REPLAY continues across a corrupt stored record so the GS can report its record-level CRC failure.

Static RTOS objects: Use static task and mutex storage where specified; do not silently introduce heap allocation.

Command gating: START only from Idle; STOP only from Recording; REPLAY and ERASE only from Idle; STATUS always.

DHT fault masks: DHT_TIMEOUT is 0x02 across firmware, tests, and ground-station decoding.

FINAL DEFINITION OF DONE Phase 0 passes; all host and Python tests pass; the complete hardware command cycle works; wraparound and recovery are demonstrated; fault-injection results are documented; and all required demo artifacts are present.